

Sistemas Multiagentes (PPGCA-CT e PPGCC-PG)

Reasoning Agents

Prof^a Celia Ghedini Ralha

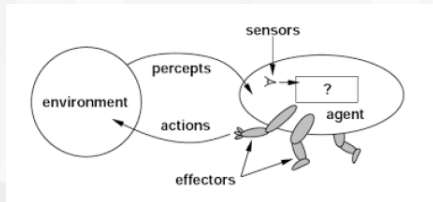
Universidade Tecnológica Federal do Paraná (UTFPR)
Curitiba/PR, Brasil

02 de outubro de 2026

Agenda

- Cognitive Agent: Properties & Applications
- Reasoning Agents
 - ▶ Symbolic
 - ▶ Reactive
 - ▶ Hybrid
- Procedural Reasoning System (PRS)
- Agents as Intentional Systems
 - ▶ BDI agents
- Agent-based Programming Languages
- MAS Open-source Frameworks
- MASPY in practice

Cognitive Agent - Properties¹



Computational entity capable of **flexible** autonomous action in some environment:

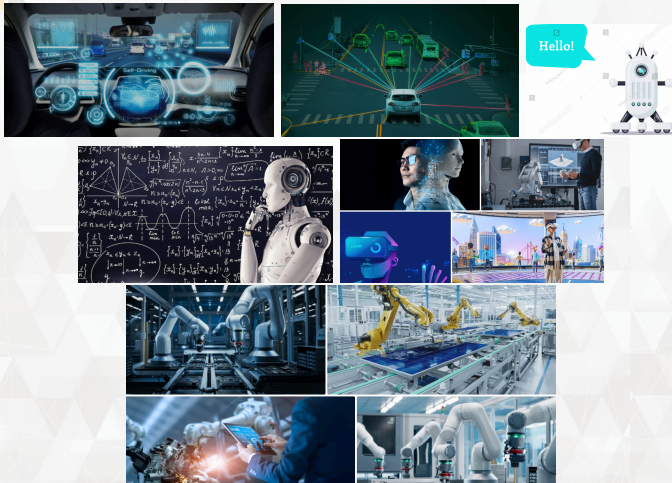
- **reactive** - with a fixed environment (static), program-driven solely by events, execute stimulus $\Rightarrow$ response rules
- **pro-active** - generating & attempting to achieve goals, take initiative, recognize opportunities (goal-oriented)
- **social** - interact with other agents (or humans) using agent-communication language, cooperate or compete

Cognitive Agent - Properties²

Other properties, sometimes discussed in the context of agency:

- **mobility** - the ability of an agent to move around an electronic network
- **veracity** - an agent will not knowingly communicate false information
- **benevolence** - agents do not have conflicting goals, and that every agent will therefore always try to do what is asked of it
- **rationality** - agent will act in order to achieve its goals, and will not act in such a way as to prevent its goals being achieved — at least insofar as its beliefs permit
- **learning/adaption** - agents improve performance over time

Cognitive Agent - Applications

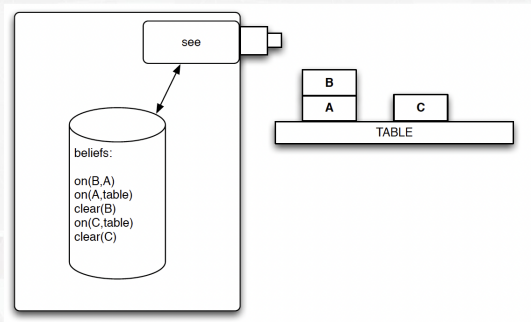


From top to bottom, left to right: autonomous driving (Waymo robotaxi), automated trading (Forex robot), automated tutoring (TutorExtra UK), commercial games (virtual reality), multi-robot factories.

Reasoning Agents³

- Symbolic (1956-today) - use logical reasoning to decide what to do, explicitly represent a symbolic model of the world (blocks world with Prolog, formal theorem-proving with Coq/Rocq)
- Reactive (1985-today) - respond directly to environmental stimuli without the use of internal symbolic models, relying on simple rules and direct interaction with their environment (Agent0)
- Hybrid (1990-today) - combine symbolic & reactive approaches (Concurrent MetateM, SOAR)

Symbolic Reasoning Agents - Blocks World



Use logic to encode a theory defining the best action to perform in any given situation. Let:

- ρ - the theory (typically a set of rules)
- Δ - logical database describing the current world state (facts)
- Ac - the set of actions the agent can perform
- $\Delta \vdash_{\rho} \phi$ mean that ϕ can be proved from Δ using ρ

Production-Rule Execution

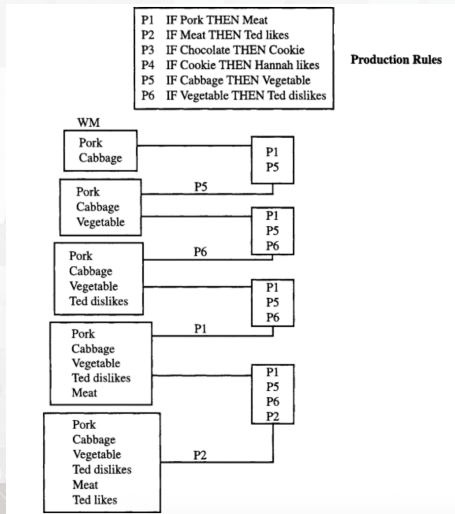
Forward Chaining^a

Rules are applied to known data to arrive at conclusions

- 1 A production rule corresponding to a variable in the working memory (WM) is selected.
- 2 A production rule with the longest conditional clause is selected.
- 3 All production rules corresponding to the variables in the WM are selected.
- 4 All production rules corresponding to the variables in the WM—except for the production rules chosen so far—are selected.

What is the conclusion?

^a Rete Algorithm (Charles L. Forgy 1974), Carnegie Mellon Univ.



Example: <https://www.sciencedirect.com/topics/computer-science/production-rule>

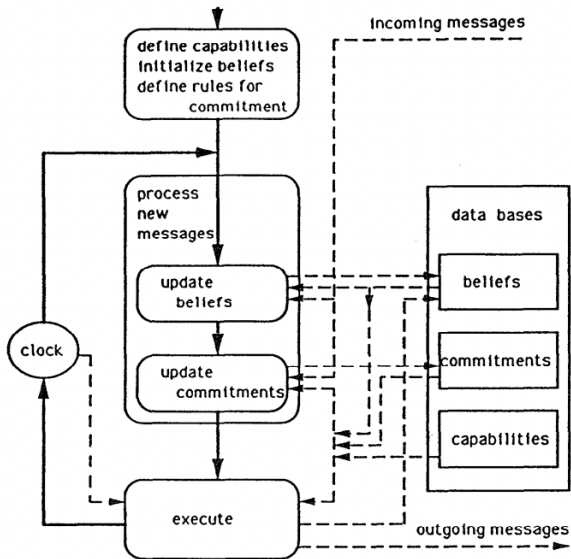
Reactive Reasoning Agents - AGENT0⁴

Y. Shoham introduced agent-oriented programming with AGENT0

- programming paradigm based on societal view of computation with 4 sets:
 - ▶ capabilities (things agent can do)
 - ▶ initial beliefs
 - ▶ initial commitments (things agent will do)
 - ▶ commitment rules - message condition, mental condition and an action

```
COMMIT(  
  ( agent, REQUEST, DO(time, action)  
  ), ;;; msg condition  
  ( B,  
    [now, Friend agent] AND  
    CAN(self, action) AND  
    NOT [time, CMT(self, anyaction)]  
  ), ;;; mental condition  
  self,  
  DO(time, action)  
)
```

AGENT0



Hybrid Agents - Concurrent MetateM⁵

Concurrent MetateM is an agent programming language that models complex reactive systems by using a combination of temporal logic specification of the behavior it should exhibit.

- Temporal logic is classical logic augmented by modal operators for describing how the truth of propositions changes over time.
- The temporal logic specifications are executed as agent behavior.

Some temporal operators

- $\square$ - it is true now, and always in the future
- $\diamond$ - it is true now, if it will be sometime in the future
- $\bigcirc$ - tomorrow (in the next state)

Example of use

- $\square$ important(agents) - means it is now, and will always be true that agents are important
- $\diamond$ important(multiagent) - means multiagent is important now, if it will be sometime in the future
- $\bigcirc$ apologise(you) - tomorrow you apologise

⁵

Fisher, M. (1993). Concurrent MetateM — A language for modelling reactive systems. In: Bode, A., Reeve, M., Wolf, G. (eds) PARLE'93 Parallel Architectures and Languages Europe. LNCS, vol 694. Springer. https://doi.org/10.1007/3-540-56891-3_15

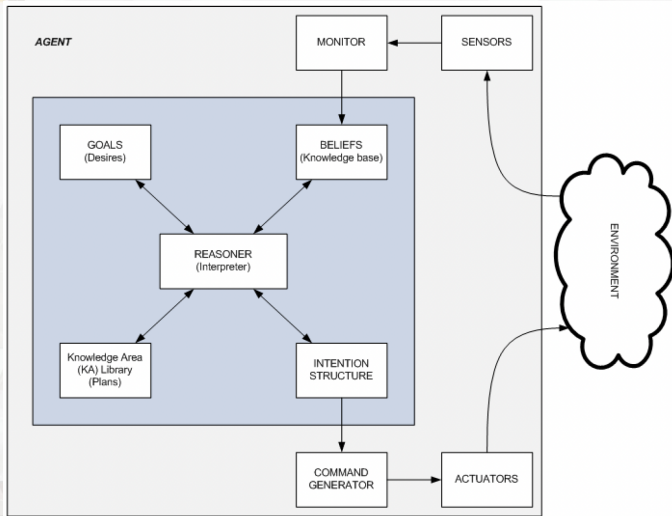
Procedural Reasoning System (PRS)

- 1st Belief-Desire-Intention practical application (explicit representation).
- Developed by AI Center at the Scientific Research Institute (SRI) International, with deep roots in Silicon Valley/USA (1980s).
- Applied by NASA in the Space Shuttle Discovery for reasoning in dynamic domains.⁶
- PRS system architecture elements:
 - ▶ BELIEFS about the world, represented using first-order predicate calculus (**Knowledge base**).
 - ▶ The system GOALS (**Desires**) as conditions over an interval of time on internal/external state descriptions.
 - ▶ Knowledge areas (KA) library (**Plans**) define sequences of low-level actions toward achieving a goal in specific situations.
 - ▶ INTENTION STRUCTURE (stack) include those KAs that have for current and eventual execution (plans that match the goal on top of the stack).
 - ▶ REASONER (**Interpreter**) or inference mechanism that manages the system.

⁶

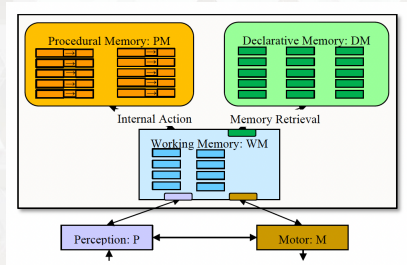
A System for Reasoning in Dynamic Domains: Fault Diagnosis on the Space Shuttle (1986). Report SRI.
<https://www.sri.com/publication/ocean-space-pubs/a-system-for-reasoning-in-dynamic-domains-fault-diagnosis-on-the-space-shuttle/>

Procedural Reasoning System (PRS)



Hybrid Agents - SOAR⁷

- State, Operator And Result (SOAR) - general cognitive architecture to create AI agents that approach human-level intelligence
 - ▶ general intelligence - from routine actions to complex
 - ▶ production rules - if-then production rules in WM
 - ▶ subgoaling - generates subgoals to solve conflicts
 - ▶ multiple memories - PM (skills), DM (facts), WM (experiences)
 - ▶ continuous learning - learns from its experience (successful problem-solving paths)



Agents as Intentional Systems

When explaining human activity, it is often useful to make statements:

- Janine took her umbrella because she **believed** it was going to rain.
- Michael worked hard because he **wanted** to possess a PhD.

These statements use **folk psychology**, by which human behavior is predicted and explained through the attribution of attitudes, mentalistic terms (e.g., believing, wanting, hoping, fearing).⁸ The attitudes employed in such descriptions are called the **intentional** notions

Daniel Dennett (philosopher) coined the term **intentional system** to describe entities 'whose behavior can be predicted by the method of attributing belief, desires and rational acumen'.^a

^a Dennett, D. C. (1971). Intentional Systems. The Journal of Philosophy, 68(4), 87–106. <https://doi.org/10.2307/2025382>

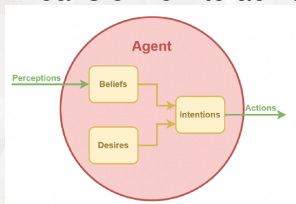
⁸ Folk Psychology as a Theory (1997) Stanford Encyclopedia of Philosophy. <https://plato.stanford.edu/entries/folkpsych-theory/>

BDI Agent

Bratman defined the theory of human practical reasoning.⁹

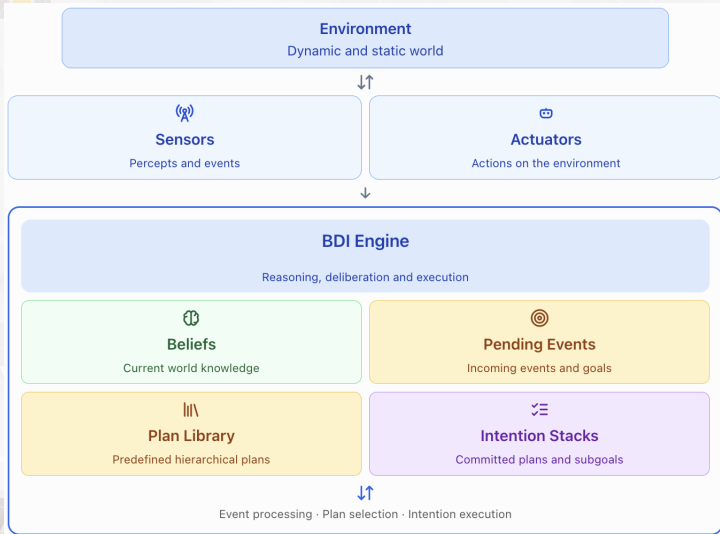
Beliefs-Desires-Intentions (BDI) are mental attitudes that represent information, motivational & deliberative states of the agent.

- **Beliefs** - knowledge about itself, other agents, current environment
- **Desires** - states the agent hopes do achieve
- **Intentions** - intended means of how to achieve the desired states



⁹ Michael Bratman (1987) *Intentions, Plans, and Practical Reason*. Harvard University Press, Cambridge, MA

Typical BDI Agent System Architecture¹⁰



¹⁰ Sardina, Sebastian; Padgham, Lin (2011). A BDI agent programming language with failure handling, declarative goals, and planning. RMIT University. Journal contribution. <https://hdl.handle.net/10779/rmit.27433407.v1>

Key Concepts - BDI Agent

- **Beliefs:** Represent the agent's knowledge about the current environment, updated from percepts and events.
- **Pending events:** Capture incoming information and goal-related events that trigger agent reasoning.
- **Plan library:** Stores predefined, context-sensitive plans associated with goals or events.
- **Intention stacks:** Maintain the selected plans and their subgoals, representing the agent's current commitments.
- **BDI engine:** Processes events, selects applicable plans, manages intentions and executes actions through actuators.

Scenario: Medication Delivery Robot



Beliefs

What the robot knows



Desires

What the robot wants



Intentions

What the robot commits to doing

Medication Delivery Robot: Execution Cycle

- Perceive the environment (Beliefs)
- Determine the goal (Desires)
- Select a plan (Intention)
- Execute the plan (Action)
- Detect an unexpected event (Event/Belief update)
- Handle failure and replan (Intention revision)
- Complete the goal (Goal achieved)

Step	Belief (B)	Desire (D)	Intention (I)
1. Initial state	Corridor clear	Deliver medication	Follow route A
2. Obstacle detected	Corridor blocked	Deliver medication	Route A suspended
3. Replanning	Alternative corridor clear	Deliver medication	Follow route B
4. Goal achieved	Medication delivered	Delivery completed	Intention dropped

BDI Agents: Central Idea

BDI = Knowing, Wanting, Committing

- **Beliefs:** What is currently true about the environment?
- **Desires:** What goals should the agent achieve?
- **Intentions:** Which plans has the agent committed to executing?
- BDI agents do not simply react to events.
- It maintains goals and commitments.
- It selects plans to achieve goals.
- Adapts its intentions when circumstances change.

Agent-based Programming Languages (APL)

APL	Model	Implementation (Language-Link)
ASTRA	BDI	Java https://gitlab.com/astra-language
Chromar	rule-based	Haskell https://github.com/azardilis/Chromar
GOAL	rule-based	Java https://goalapl.atlassian.net/wiki/spaces/GOAL/overview
Gwendolen	BDI	Java https://github.com/mcapl/mcapl
JaCaMo	BDI, organisation, environment	Java https://github.com/jacamo-lang/jacamo
JADE	FIPA	Java https://jade.tilab.com/
JADEL	DSL, interaction	Java/Jade ✗
Jadex	mixed, BDI and OOP	Java https://github.com/actoron/jadex
APL	Model	Implementation (Language-Link)
Jadescript	DSL, scripting	Java/Jade ✗
Jason	BDI	Java https://github.com/jason-lang/jason
LightJason	BDI	Java https://github.com/LightJason/
PLACE	BDI, HTN	✗
PLASA	Wait-Look-Compute-Move	Java ✗
RMAS	database-centric, CPS	Matlab/SQLite ✗
SARL	DSL	Java https://github.com/sarl/sarl

MAS Open-source Frameworks

- JADE[†] - **J**ava **A**gent **D**Evelopment¹¹
- JADEX - **J**ava **A**gent **D**Evelopment **E**xtended (JADE + BDI model)¹²
- SPADE[†] - **S**mart **P**ython **A**gent **D**evelopment **E**nvironment¹³
- MASPY - **M**ulti-**A**gent **S**ystem for **P**Ython¹⁴
- PADE[†] - **P**ython **A**gent **D**Evelopment¹⁵

[†] Compliant with Foundation for Intelligent Physical Agents (FIPA) standards

¹¹ <https://jade.tilab.com/>. Fabio L. Bellifemine, Giovanni C. & Dominic Greenwood. Developing Multi-Agent Systems with JADE, Wiley Series in Agent Technology, 2007.

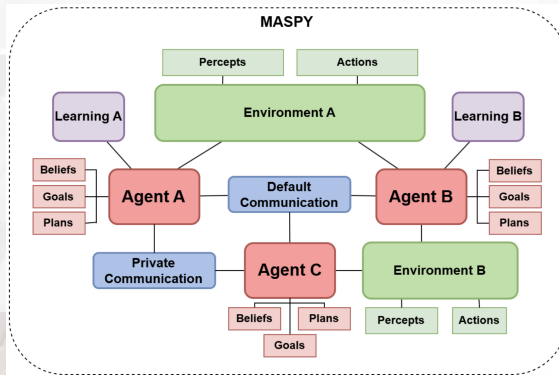
¹² Distributed & Information Systems Group, Univ. of Hamburg, Germany <https://sourceforge.net/projects/jadex/>
¹³ <https://spadeagents.eu/>

¹⁴ Prof Gleifer Vaz Alves & LaCA research group (gleifer,mellado,apborges)@utfpr.edu.br
<https://github.com/laca-is/MASPY/wiki>

¹⁵ <https://github.com/grei-ufc/pade>

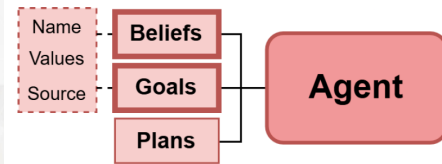
Five main classes:

- Admin class - system's configuration & control
- Agent Class - manage beliefs, goals & plans
- Environment Class - provide perception & interaction context to agents
- Communication Class - agents exchange messages & information
- Learning Class - model environment instance into ML-ready training



MASPY - Beliefs & Goals

- Beliefs represent agent's knowledge about the environment
- Goals represent the achievement of agent's intention in environment
- Beliefs & Goals can be acquired internally (self) or by messages from other agents¹⁶



Belief and Goal Identifier

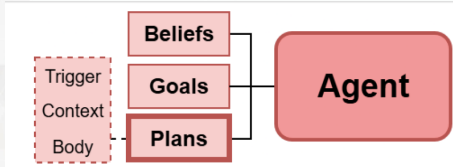
	Name	Values	Source
Belief	"spot"	(12, "free")	"Parking"
Belief	"Receiver"		"self"
Goal	"SendSpot"	"Driver_5"	"self"
Goal	"receive_info"	message	"Sender"

Belief & Goal syntax.

¹⁶ Perception are always new beliefs

MASPY - Events & Plans

- Events are agent's information changes (beliefs or goals)
- Such changes can be related to gaining/losing/testing information
- Plans are activated though triggers
- Plans are defined by inserting a **@pl** decoration in an agent's function



```

Plan Identifier
↑
@pl (gain, Goal("receive_info", Any), Belief("receiver"))
def recv_info(self, src, msg):
    self.print(f"Information [{msg}] - Received from {src}")
    self.send(src, tell, Belief("info_received"))
    self.stop_cycle()
    
```

Trigger Event Context Body

Plan syntax.

MASPY - Percepts & Actions

- Percepts represent environment characteristics observable by connected agents
- Perceptions are translated into agent's beliefs



Percept Identifier

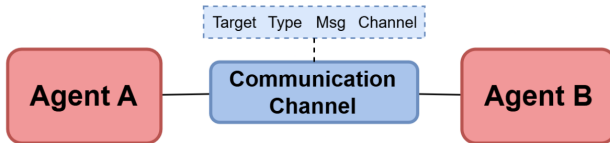
	Name	Values	Group
Percept	"spot"	(12, "free")	"Spots"
Belief	"spot"	(12, "free")	"Parking"

Perception

Percept syntax.

MASPY - Communication Channel

- Agents start connected to default communication channel, but can connect to more than one channel
- Messages can be send to one agent or broadcast to all agents in the channel
- Message content have to be aligned to the performative type used with one or a list of data



Function to send message syntax.

MASPY - Communication Language

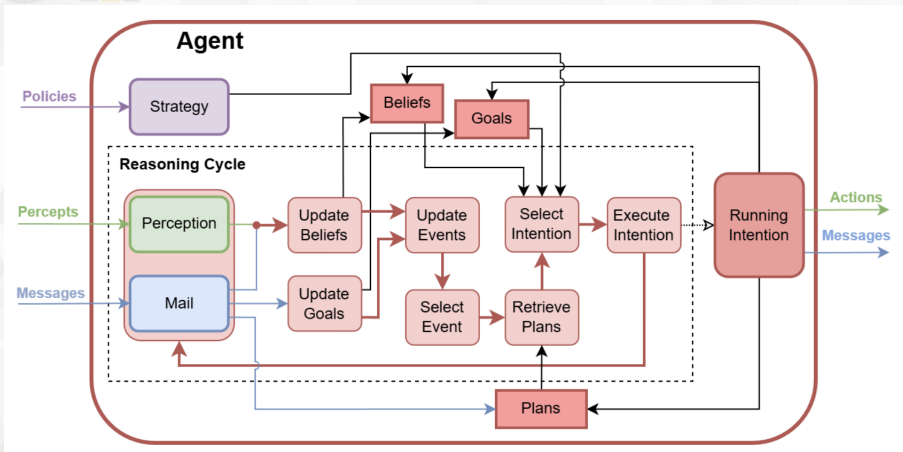
- A KQML-based (Knowledge Query Model Language) is used to communication between agents
- Using KQML performatives, agents can ask, tell, command, teach their needs and knowledge

```
self.send(<target>, <directive>, <info>, optional[<channel>])
```

Directives:

tell	→ Add Belief on target
untell	→ Remove Belief from target
achieve	→ Add Goal to target
unachieve	→ Remove Goal from target
askOne	→ Ask for Belief from target
askOneReply	→ Ask for Belief from target and wait for Reply
askAll	→ Ask for all similar Beliefs from target
askAllReply	→ Ask for all similar Beliefs from target and wait for Reply
tellHow	→ Add Plan on target
untellHow	→ Remove Plan from target
askHow	→ Ask for Plan from target
askHowReply	→ Ask for Plan from target and wait for Reply

MASPY - BDI Agent Reasoning Cycle¹⁷



¹⁷ Mellado, Alexandre L. L.; Fidler, Igor G.; Borges, André P.; Alves, Gleifer V. MASPY: Towards the Creation of BDI Multi-Agent Systems. In: Workshop-Escola de Sistemas de Agentes, Seus Ambientes e Aplicações (WESAAC), 2023, Pelotas/RS. p. 106-117. DOI: 10.5753/wesaac.2023.33440

To Install MASPY

- Use Python Version ≥ 3.12 (64 bits)
- Pip Install maspy-ml
- code to test <tinyurl.com/maspy>
- Execution Console:

```
# Admin #> Starting MASPY Program - ver.2026.05.13
# Admin #> Starting All Agents
# Admin #> Starting System
Agent:HelloAgent> Hello World!
# Admin #> Closing System
# Admin #> Ending MASPY Program
```

Hello World

MASPY/examples/ex-hello-world.py

(<https://github.com/laca-is/MASPY/blob/main/examples/ex-hello-world.py>)

```
1 from maspy import *
2
3 class HelloAgent(Agent):
4     @pl(gain, Belief("hello"))
5     def func(self, src):
6         self.print(f"Hello_World!")
7         self.stop_cycle()
8
9 if __name__ == "__main__":
10     ag = HelloAgent(beliefs=Belief("hello"))
11     Admin().start_system()
```

Scenario: BDI Agent Move Boxes in MASPY

- Two autonomous agents (Dummy_1, Dummy_2) have the goal move boxes.
- Each agent has its own initial position and target position, both have a box at (5,10).
- The agents use a goal-triggered plan to pick up the box & move it to their respective target positions.

Environment (grid world)

Dummy_1

Beliefs:

- my_pos = (0,0)
- target_pos = (7,7)
- Box = (5,10)

Goal:

- move_boxes



Dummy_2

Beliefs:

- my_pos = (3,3)
- target_pos = (3,3)
- Box = (5,10)

Goal:

- move_boxes

Move Boxes

MASPY/examples/ex-move-boxes.py

(<https://github.com/laca-is/MASPY/blob/main/examples/ex-move-boxes.py>)

```

1 from maspy import *
2 class DummyAgent(Agent):
3     def __init__(self, name, beliefs, goals):
4         super().__init__(name, beliefs, goals, show_exec=False)
5         self.add(Belief("Box", (5, 10)))
6
7     @pl(gain, Goal("move_boxes"), Belief("Box", (Any, Any)))
8     def move_to_pos(self, src, position):
9         x, y = position
10        my_pos = self.get(Belief("my_pos", (Any, Any)))
11        self.move(my_pos.values, (x, y))
12        self.print(f"Picking_up_Box_in_{x,y}")
13        target_pos = self.get(Belief("target_pos", (Any, Any)))
14        self.move((x, y), target_pos.values)
15        self.print(f"Putting_Box_in_{target_pos.values}")
16        self.stop_cycle()
17
18    def move(self, my_pos, target_pos):
19        self.print(f"Moving_from_{my_pos}_to_target_{target_pos}_position")
20
21 if __name__ == "__main__":
22     agent_1 = DummyAgent("Dummy_1", [Belief("my_pos", (0, 0)), Belief("target_pos", (7, 7))],
23                               Goal("move_boxes"))
24     agent_2 = DummyAgent("Dummy_2", [Belief("my_pos", (3, 3)), Belief("target_pos", (3, 3))],
25                               Goal("move_boxes"))
26     Admin().start_system()

```

Your Move Boxes

MASPY/examples/ex-move-boxes.py

(<https://github.com/laca-is/MASPY/blob/main/examples/ex-move-boxes.py>)

Include 1 new agent -> Dummy_3 with its own initial position and target position

MASPY Framework

Documentation

- Home: <https://github.com/laca-is/MASPY>
- Wiki: <https://github.com/laca-is/MASPY/wiki>
- GUI: <https://github.com/laca-is/MASPY-GUI>

Publications

- Alexandre L. L. Mellado, Gabriel G. Neres, André P. Borges, Rafael C. Cardoso, and Gleifer V. Alves. 2026. MASPY: A Python Framework for Developing BDI Agents with Reinforcement Learning. In Proceedings of the 25th International Conference on Autonomous Agents and Multiagent Systems (AAMAS'26). IFAAMAS, Richland, SC, 4074–4076. <https://doi.org/10.65109/YIGW5980>
- Mellado, A.L.L., Pinz Borges, A., Alves, G.V. (2026). MASPY: A Python-Based Framework for Developing BDI Multi-agent Systems. In: Mathieu, P., De la Prieta, F. (eds) Advances in Practical Applications of Agents, Multi-Agent Systems, and Computational Social Science: The PAAMS Collection. PAAMS Workshop 2025. LNCS, vol 16031. Springer, Cham. https://doi.org/10.1007/978-3-032-07638-0_18

Summary

- Cognitive Agent: Properties & Applications
- Reasoning Agents
 - ▶ Symbolic - blocks
 - ▶ Reactive - AGENT0
 - ▶ Hybrid - Concurrent MetateM, SOAR
- Procedural Reasoning Systems
- Agents as Intentional Systems
 - ▶ BDI Agents
- Agent-based Programming Languages
- MAS Open-Source Frameworks
- MASPY Theory & Practice -
 - ▶ 5 classes, Beliefs & Goals, Events & Plans, Percepts & Actions, Communication Channel & Language, Reasoning Cycle
 - ▶ MASPY intallation
 - ▶ Examples: Hello World & Move Boxes

Thank you for coming!



Prof^a Celia Ghedini Ralha
celiaghedini@utfpr.edu.br